

Instituto Tecnológico Autónomo de México

Cómputo Paralelo y en la Nube

Descripción del código y de las estrategias de paralelización

DBSCAN paralelo para la detección de ruido o outliers con OpenMP

Curso	Cómputo Paralelo y en la Nube
Profesor	Dr. J. Octavio Gutiérrez García
Equipo	Marcio Tellez e Irene Escudero
Entrega	20 de octubre de 2026
Repositorio	github.com/TellezM/Proyecto_Par_y_Nube

1. Qué hace el programa

El programa recibe puntos en el plano y decide cuáles son ruido (outliers) usando la idea de densidad de DBSCAN. No arma los clusters: solo etiqueta cada punto, que es lo que pide el proyecto (el "algoritmo ingenuo" de las diapositivas). Hay dos parámetros, que son los mismos de la libreta `DBSCAN_noise.ipynb`: la distancia épsilon (`eps = 0.03`) y el número mínimo de puntos para que una zona se considere densa (`min_samples = 10`).

Cada punto queda en una de tres categorías:

Tabla 1. *Etiquetas que escribe el programa en la tercera columna del CSV de salida.*

Etiqueta	Nombre	Cuándo se asigna
1	Core de primer orden	Tiene al menos `min_samples` puntos a distancia ≤ `eps` (se cuenta a sí mismo, igual que `sklearn`).
2	Core de segundo orden	No es core, pero hay un core de primer orden a distancia ≤ `eps`.
0	Ruido (outlier)	No cumple ninguna de las dos condiciones anteriores.

La entrada es un archivo CSV con dos columnas `x,y` sin encabezado (se genera con la misma receta de la libreta) y la salida es otro CSV con `x,y,etiqueta`, que la libreta puede graficar. La Figura 1 muestra el resultado del programa sobre 4,000 puntos junto al de `sklearn.DBSCAN`: el ruido es el mismo.

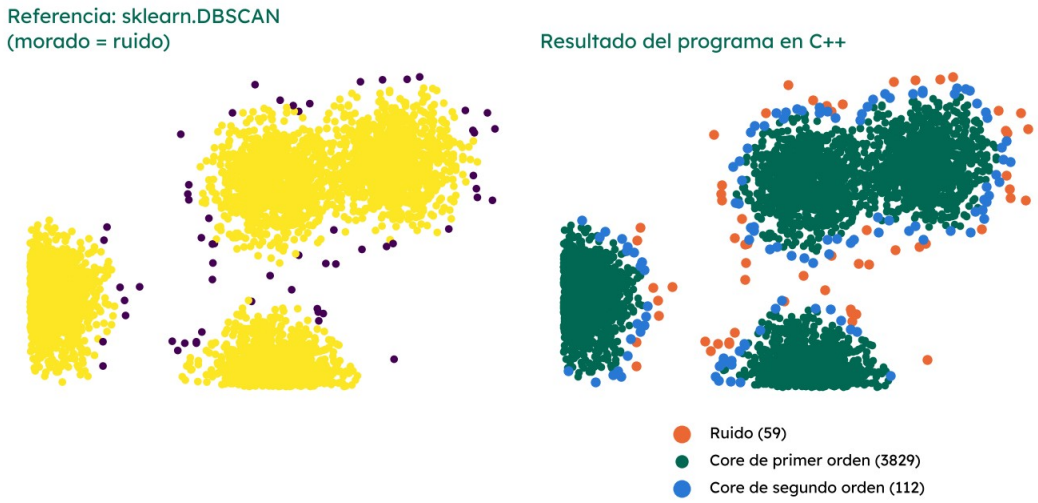


Figura 1. *Izquierda: referencia con sklearn (el ruido en morado). Derecha: resultado del programa en C++ (versión P2, k=16) sobre los mismos 4,000 puntos.*

2. Archivos y cómo usarlos

Tabla 2. *Archivos del repositorio.*

Archivo	Para qué sirve
`proy_ruido.cpp`	Todo el código del algoritmo: lectura y escritura de CSV, versión serial, P1 y P2. Está comentado en español.
`scripts/generar_datos.py`	Genera los CSV de entrada con la misma receta de la libreta (4 blobs, `cluster_std=0.06`, `random_state=11`).
`scripts/correr_experimento.py`	Corre el experimento completo de la rúbrica y guarda `resultados_tiempos.csv`. También comprueba en cada corrida que la salida sea igual a la de la versión serial.
`scripts/analizar_resultados.py`	Promedia las iteraciones, calcula speedups y hace las gráficas.
`estudio_decision/`	Código del estudio previo con el que se decidió cómo paralelizar (sección 6): una copia del programa con las variantes que se compararon (`proy_ruido_estudio.cpp`) y sus scripts.
`scripts/prueba_correctitud.py`	Compara P1 y P2 contra la versión serial y contra `sklearn.DBSCAN`.
`DBSCAN_noise.ipynb`	Libreta del curso (generar datos y graficar). Se actualizó la parte de speedups.
`resultados/`	Datos: tiempos de todas las corridas (`resultados_tiempos.csv`), resumen con speedups y los datos del estudio de decisión.
`imagenes/`	Gráficas y capturas que aparecen en los documentos.
`entregables/`	Los dos documentos de la entrega (Word y PDF).

Para compilar basta `g++` con OpenMP, como en las diapositivas de OpenMP (en Windows con MSYS2/MinGW desde VS Code, o en Linux):

Compilación

```
g++ -O2 -fopenmp proy_ruido.cpp -o dbscan
```

Ejecución (en Windows: `.\dbscan.exe`)

```
./dbscan <entrada.csv> <salida.csv> <eps> <min_samples> <version> <k> <hilos>

# ejemplo: P2 con 16 regiones y 4 hilos
./dbscan datos/4000_data.csv salida.csv 0.03 10 p2 16 4
```

`version` puede ser `serial`, `p1` o `p2`. El programa imprime un solo número, el tiempo del algoritmo en segundos medido con `omp_get_wtime()`. En ese tiempo no entra la lectura ni la escritura de los CSV, pero sí entra la construcción de la cuadrícula de P2, porque es parte del algoritmo. `k` solo lo usa P2 y `hilos` no afecta a la versión serial.

3. Versión serial

El algoritmo ingenuo tiene dos fases. En la primera se recorre cada punto `i` y se cuentan los puntos `j` que están a distancia ≤ `eps`; si son al menos `min_samples`, el punto es core. En la segunda, cada punto que no es core busca algún core a distancia ≤ `eps`: si lo encuentra es de segundo orden y si no, es ruido. La búsqueda de la segunda fase se detiene en cuanto encuentra el primero.

dbscan_serial (resumen)

```
// Fase 1: cores de primer orden
for (int i = 0; i < n; ++i) {
    int vecinos = 0;
    for (int j = 0; j < n; ++j)
        if (dist2(x[i], y[i], x[j], y[j]) <= eps2) vecinos++;
    core[i] = (vecinos >= min_samples);
}
// Fase 2: cores de segundo orden y ruido
for (int i = 0; i < n; ++i) {
    if (core[i]) { labels[i] = 1; continue; }
    bool es_borde = false;
    for (int j = 0; j < n; ++j)
        if (core[j] && dist2(x[i], y[i], x[j], y[j]) <= eps2) { es_borde = true; break; }
    labels[i] = es_borde ? 2 : 0;
}
```

Dos detalles de implementación: se compara la distancia al cuadrado contra `eps*eps` para no calcular raíces cuadradas, y el resultado de la fase 1 se guarda en un arreglo aparte (`core`) en lugar de usar el mismo arreglo de etiquetas. Esto último importa en las versiones paralelas, como se explica en las secciones 4 y 5. El costo es $O(n^2)$, porque cada punto se compara con todos los demás.

4. Versión paralela P1: matriz completa

En P1 cada hilo trabaja sobre toda la matriz de puntos sin dividirla. Se paraleliza el ciclo exterior de cada fase con `#pragma omp parallel for`, de modo que cada hilo se encarga de un conjunto de puntos `i` y los compara contra todos los demás.

dbscan_p1 (resumen)

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < n; ++i) {            // Fase 1
    int vecinos = 0;
    for (int j = 0; j < n; ++j)
        if (dist2(x[i], y[i], x[j], y[j]) <= eps2) vecinos++;
    core[i] = (vecinos >= min_samples);
}                                        // barrera implícita
#pragma omp parallel for schedule(static)
for (int i = 0; i < n; ++i) { ... }      // Fase 2, igual que en la serial
```

Por qué es correcto. Cada iteración `i` lee `x`, `y` y `core` y escribe únicamente `core[i]` (fase 1) o `labels[i]` (fase 2), así que dos hilos nunca escriben la misma posición. No hacen falta `critical`, `atomic` ni `reduction`. Entre las dos fases hay una barrera implícita al final del primer `parallel for`: cuando empieza la fase 2, `core` ya está completo. Si la fase 2 leyera y escribiera el mismo arreglo, un hilo podría leer la posición de un punto justo cuando otro la está

cambiando; por eso se separaron `core` (que en la fase 2 solo se lee) y `labels` (que solo se escribe).

Qué ciclo se paralelizó y con qué planificación. Se paralelizó el exterior y se dejó `schedule(static)`. Las dos decisiones se tomaron con datos (secciones 6.1 y 6.2): paralelizar el interior es mucho peor y, con este trabajo, las distintas planificaciones rinden igual.

5. Versión paralela P2: matriz dividida en k regiones

En P2 el plano se divide en `k` regiones rectangulares (una cuadrícula de `filas × columnas = k`, lo más cuadrada posible: k=2 es 1×2, k=4 es 2×2, k=8 es 2×4, k=16 es 4×4, k=32 es 4×8 y k=64 es 8×8). Primero se calcula el rectángulo que contiene a todos los puntos y se corta en celdas iguales. Después los puntos se reordenan por región con un ordenamiento por conteo (cuesta O(n)): los puntos de la región `c` quedan en un bloque contiguo de memoria, en las posiciones `inicio[c]` a `inicio[c+1]-1` de los arreglos `xs` y `ys`. Se guarda también `orden[p]`, el índice original de cada punto, para regresar las etiquetas al orden de entrada al final. Esta es la forma de "dividir la matriz" que usa la versión final; la primera versión guardaba solo listas de índices y la cambiamos con datos (sección 6.5).

Qué compara cada región. Para saber si un punto de una región es core no basta mirar su propia región, porque un vecino a distancia ≤ `eps` puede estar en una celda de al lado. Por eso cada punto se compara con los puntos de su región y de las regiones vecinas (el "halo"). El halo se calcula en celdas: `ceil(eps / tamaño_de_celda)` hacia cada lado. En el código original el halo era siempre de una celda, y eso solo es correcto si cada celda mide al menos `eps`; con los datos del proyecto sí se cumple, pero lo generalizamos para que el resultado sea correcto con cualquier `k` y cualquier `eps` (lo comprobamos con `eps = 0.30`, donde el halo es de varias celdas). Como las celdas de una misma fila de la cuadrícula quedan una tras otra en memoria, el halo de cada fila de celdas es un solo rango contiguo.

rangos_vecinos

```
// Rangos de posiciones que hay que revisar para la región r_id: una por cada fila de celdas
del halo
void rangos_vecinos(const CuadriculaOrd& g, int r_id, vector<pair<int,int>>& out) {
    out.clear();
    int r_row = r_id / g.cols, r_col = r_id % g.cols;
    int c0 = max(0, r_col - g.halo_c), c1 = min(g.cols - 1, r_col + g.halo_c);
    for (int dr = -g.halo_r; dr <= g.halo_r; ++dr) {
        int nr = r_row + dr;
        if (nr < 0 || nr >= g.rows) continue;
        out.push_back({g.inicio[nr * g.cols + c0], g.inicio[nr * g.cols + c1 + 1]});
    }
}
```

dbscan_p2, fase 1 (resumen)

```
#pragma omp parallel for schedule(dynamic, 1)
for (int r_id = 0; r_id < k; ++r_id) {          // un hilo toma una región completa
    vector<pair<int,int>> vec;  rangos_vecinos(g, r_id, vec);
    for (int p = g.inicio[r_id]; p < g.inicio[r_id + 1]; ++p) {
        int vecinos = 0;
        for (auto& rg : vec)
            for (int q = rg.first; q < rg.second; ++q)
                if (dist2(g.xs[p], g.ys[p], g.xs[q], g.ys[q]) <= eps2) vecinos++;
        core[p] = (vecinos >= min_samples);      // solo escribe puntos de su región
    }
}                                                // barrera implícita
```

Cómo se unen los resultados para mantener la consistencia. Es la pregunta que deja abierta el proyecto y aquí no hace falta un paso de unión aparte, por dos razones. Primero, las regiones son una partición: cada punto pertenece a una sola región, así que cada hilo escribe solo los puntos de su región y los halos se usan únicamente para leer. Segundo, entre la fase 1 y la fase 2 hay una barrera: cuando empieza la fase 2 ya se sabe, para todos los puntos de todas las regiones, cuáles son core. Un punto que está en el borde de una región puede entonces ser clasificado como de segundo orden gracias a un core de la región de al lado, igual que en la versión serial. Por eso P2 da exactamente el mismo resultado que la serial (sección 7).

Planificación y límite de paralelismo. Las regiones no tienen el mismo número de puntos (los datos están en cuatro grupos), así que se reparten con `schedule(dynamic, 1)`: cada hilo que se desocupa toma la siguiente región (sección 6.3). El paralelismo de P2 está limitado por `k`: con k=2 solo dos hilos tienen trabajo. Además, la parte que reordena los puntos y construye la cuadrícula es serial, pero cuesta O(n) frente al resto del algoritmo, que es de orden n² (dividido entre el número de regiones que se descartan).

De dónde sale la mejora de P2. Hay dos efectos distintos y conviene no mezclarlos. Uno es de algoritmo: dividir en regiones evita comparar cada punto contra todos los demás, aun con un solo hilo. El otro es de paralelismo: repartir las regiones entre hilos. En el documento de evaluación se miden por separado (P2 con 1 hilo contra la serial, y P2 con varios hilos contra P2 con 1 hilo).

6. Cómo decidimos el método de paralelización

Antes de correr el experimento completo hicimos un estudio previo con la misma entrada de la rúbrica para decidir los puntos donde había más de una opción razonable. Cada configuración se corrió 5 veces; las tablas muestran la media y la desviación estándar del tiempo en segundos y el speedup contra la versión serial con la misma entrada. Los datos están en `estudio_decision.csv` y se reproducen con `estudio_decision.py`.

6.1 ¿Qué ciclo paralelizar?

Tabla 3. *P1 paralelizando el ciclo exterior o el interior con* `reduction` *(20,000 puntos, serial = 0.40 s).*

Hilos	Ciclo exterior: tiempo (s)	Speedup	Ciclo interior: tiempo (s)	Speedup
2	0.22 ± 0.01	1.81	0.34 ± 0.13	1.16

Hilos	Ciclo exterior: tiempo (s)	Speedup	Ciclo interior: tiempo (s)	Speedup
4	0.20 ± 0.01	2.02	1.25 ± 0.07	0.32

Paralelizar el ciclo interior obliga a abrir y cerrar una región paralela por cada punto (20,000 veces por fase) y a combinar un contador con `reduction`. Con 2 hilos todavía mejora algo, pero con 4 hilos es 3.1 veces más lento que la serial (speedup 0.32). Paralelizar el ciclo exterior reparte trabajo grueso, sin sincronización, y es la opción que usamos en P1. Es la situación de las diapositivas de OpenMP sobre "fors anidados": con ciclos anidados conviene paralelizar el exterior.

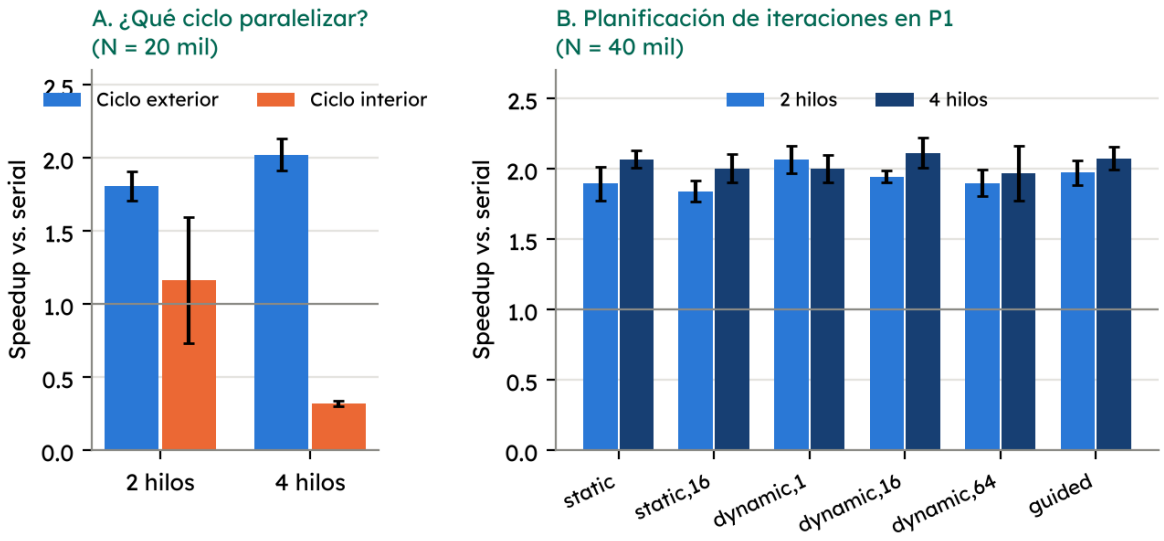


Figura 2. *Izquierda: ciclo exterior contra interior en P1. Derecha: planificación de las iteraciones en P1. Las barras de error son la desviación estándar de las 5 corridas.*

6.2 Planificación de las iteraciones en P1

Tabla 4. *Planificación de P1 con 40,000 puntos (serial = 1.73 s).*

Planificación	2 hilos: tiempo (s)	Speedup	4 hilos: tiempo (s)	Speedup
static	0.91 ± 0.06	1.89	0.84 ± 0.02	2.07
static,16	0.94 ± 0.04	1.84	0.86 ± 0.04	2.00
dynamic,1	0.84 ± 0.04	2.06	0.87 ± 0.04	2.00
dynamic,16	0.89 ± 0.02	1.94	0.82 ± 0.04	2.11
dynamic,64	0.91 ± 0.04	1.90	0.88 ± 0.09	1.97
guided	0.88 ± 0.04	1.97	0.83 ± 0.03	2.07

Con 2 hilos la planificación más rápida fue `dynamic,1` y con 4 hilos `dynamic,16`. La diferencia entre la mejor y la peor es de 11% con 2 hilos y de 7% con 4, y la variación entre corridas de una misma configuración es de 5% en promedio. Como la mejor cambia de un número de hilos a

otro y las diferencias son pequeñas, no hay una planificación claramente mejor. Esto tiene sentido: en la fase 1 todas las iteraciones hacen exactamente el mismo trabajo (comparar contra los n puntos) y en la fase 2 casi todos los puntos son core y salen de inmediato, así que repartir las iteraciones en bloques iguales (`static`) ya queda balanceado. Elegimos `schedule(static)` porque es la más simple y no tiene costo de reparto en tiempo de ejecución. Habría que cambiar de opinión si las iteraciones tuvieran costos muy distintos.

6.3 Planificación de las regiones en P2

Tabla 5. *Planificación de las regiones en P2 con 80,000 puntos (serial = 6.89 s).*

k	Planificación	2 hilos: tiempo (s)	Speedup	4 hilos: tiempo (s)	Speedup
4	static	3.70 ± 0.11	1.86	3.52 ± 0.11	1.96
4	dynamic,1	4.10 ± 0.16	1.68	3.59 ± 0.12	1.92
4	guided	3.69 ± 0.17	1.87	3.60 ± 0.15	1.91
16	static	1.75 ± 0.03	3.94	1.77 ± 0.08	3.90
16	dynamic,1	1.73 ± 0.08	3.99	1.75 ± 0.06	3.93
16	guided	1.73 ± 0.09	3.99	1.64 ± 0.07	4.20
64	static	0.82 ± 0.03	8.40	0.91 ± 0.05	7.55
64	dynamic,1	0.84 ± 0.03	8.18	0.87 ± 0.02	7.91
64	guided	0.83 ± 0.07	8.31	0.90 ± 0.05	7.70

Con los puntos de cada región guardados de forma contigua, las tres planificaciones dan resultados casi iguales. Con k=64 y 2 hilos el speedup es 8.40 con `static`, 8.18 con `dynamic,1` y 8.31 con `guided`; con 4 hilos, 7.55, 7.91 y 7.70. En todas las combinaciones de la tabla la diferencia entre la mejor y la peor planificación es de 11% o menos, y la variación entre corridas de una misma configuración es de 4% en promedio, así que estos datos no permiten elegir una por rendimiento. Dejamos `schedule(dynamic, 1)` por una razón de diseño y no porque las mediciones lo pidan: las regiones no tienen el mismo número de puntos y, con otros datos (por ejemplo un grupo muy denso y los demás casi vacíos), un reparto fijo podría dejar a un hilo con mucho más trabajo que a otro. Con `dynamic, 1` cada hilo toma la siguiente región cuando se desocupa, y como `k` es a lo más 64 el costo de ese reparto es insignificante. Es una decisión que habría que revisar con otras distribuciones de puntos.

C. Planificación de las regiones en P2 (N = 80 mil)

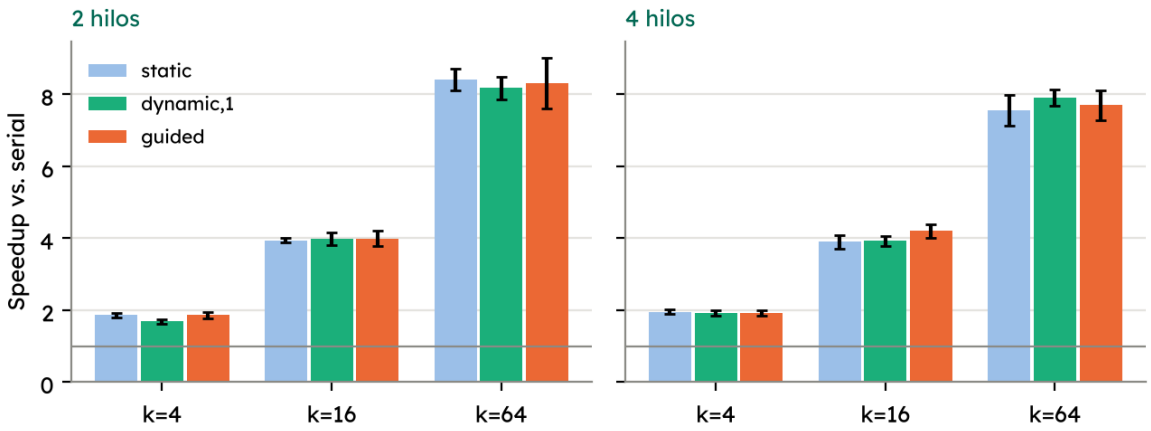


Figura 3. *Planificación de las regiones en P2 con los puntos reordenados por región (80,000 puntos), con 2 y 4 hilos.*

6.4 Paralelizar por región o por punto en P2

Tabla 6. *Misma cuadrícula de listas de índices, con el paralelismo repartido por región o por punto (80,000 puntos).*

k	Hilos	Por región: tiempo (s)	Speedup	Por punto: tiempo (s)	Speedup
4	2	5.78 ± 0.28	1.19	4.75 ± 0.08	1.45
4	4	5.12 ± 0.07	1.35	4.84 ± 0.13	1.42
16	2	2.76 ± 0.14	2.49	2.80 ± 0.14	2.47
16	4	3.12 ± 0.16	2.21	2.82 ± 0.22	2.44
64	2	1.53 ± 0.10	4.50	1.71 ± 0.12	4.04
64	4	1.58 ± 0.06	4.35	1.61 ± 0.11	4.27

Con la misma cuadrícula se puede repartir el trabajo por región (un hilo toma una región completa) o por punto (el `parallel for` recorre todos los puntos y cada uno busca en su región y su halo). Con k=16 y k=64 son equivalentes (la mayor diferencia es de 11%). Con pocas regiones, por punto gana un poco (k=4: 4 regiones desiguales no se pueden repartir bien entre hilos), pero aun así queda lejos de lo que logran los k grandes. Nos quedamos con el reparto por región porque es lo que pide el proyecto (dividir la matriz en k partes y estudiar el efecto de k) y porque hace que k sea realmente el parámetro de la estrategia. Queda anotado que el reparto por punto es una opción para k chico.

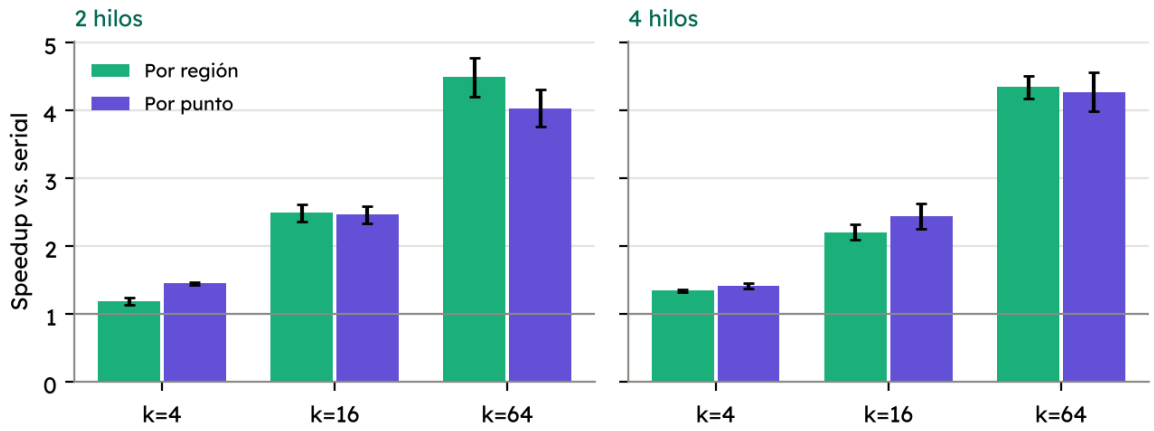


Figura 4. *Reparto por región contra por punto, con listas de índices (80,000 puntos).*

6.5 Cómo se guardan los puntos de cada región

Tabla 7. *P2 con 2 hilos: puntos de cada región como listas de índices o reordenados por región en bloques contiguos.*

k	Puntos	Listas de índices: tiempo (s)	Speedup	Reordenado: tiempo (s)	Speedup	Mejora
4	20,000	0.29 ± 0.01	1.39	0.24 ± 0.02	1.69	1.21
4	40,000	1.28 ± 0.11	1.35	1.04 ± 0.07	1.67	1.23
4	80,000	5.76 ± 0.11	1.20	4.01 ± 0.04	1.72	1.44
4	120,000	17.46 ± 0.48	0.87	9.57 ± 0.25	1.59	1.83
16	20,000	0.17 ± 0.01	2.40	0.12 ± 0.01	3.23	1.35
16	40,000	0.60 ± 0.05	2.88	0.39 ± 0.05	4.39	1.52
16	80,000	2.80 ± 0.20	2.46	1.65 ± 0.04	4.18	1.70
16	120,000	10.77 ± 0.51	1.41	3.82 ± 0.16	3.98	2.82
64	20,000	0.09 ± 0.01	4.67	0.08 ± 0.02	5.25	1.12
64	40,000	0.38 ± 0.04	4.56	0.23 ± 0.03	7.46	1.64
64	80,000	1.78 ± 0.25	3.87	0.95 ± 0.07	7.24	1.87
64	120,000	6.37 ± 0.34	2.39	1.97 ± 0.16	7.71	3.23

La primera versión de P2 guardaba, para cada región, la lista de índices de sus puntos y leía las coordenadas en el orden original. En una corrida piloto del experimento completo, el speedup de esa versión bajaba al aumentar el número de puntos aunque `k` no cambiara, lo que no tiene sentido si lo único que cambia es el tamaño del problema. Lo que sospechamos es memoria caché: con índices, cada comparación lee `x[j]` y `y[j]` en posiciones dispersas, y cuando los arreglos de coordenadas ya no caben en la caché L2 de cada núcleo muchas lecturas son fallos

de caché. En el equipo del experimento la L2 es de 1 MiB por núcleo (2 MiB en total para los 2 núcleos) y `x` e `y` ocupan 16 bytes por punto, así que dejan de caber a partir de unos 65,000 puntos, que es justo donde aparece la caída. La versión serial, en cambio, recorre los arreglos de forma secuencial. No medimos contadores de hardware para confirmarlo, pero la prueba de las dos versiones lado a lado muestra que reordenar los puntos por región las mejora en todos los casos y que la mejora crece con el tamaño de la entrada.

La segunda versión copia los puntos a arreglos ordenados por región. Es una copia de O(n), que se cuenta dentro del tiempo medido, y a cambio cada región recorre bloques contiguos de memoria. Además el halo de una fila de celdas se convierte en un solo rango. Dejamos esta versión como P2 final y conservamos la anterior como la variante `p2idx` para poder repetir esta comparación.

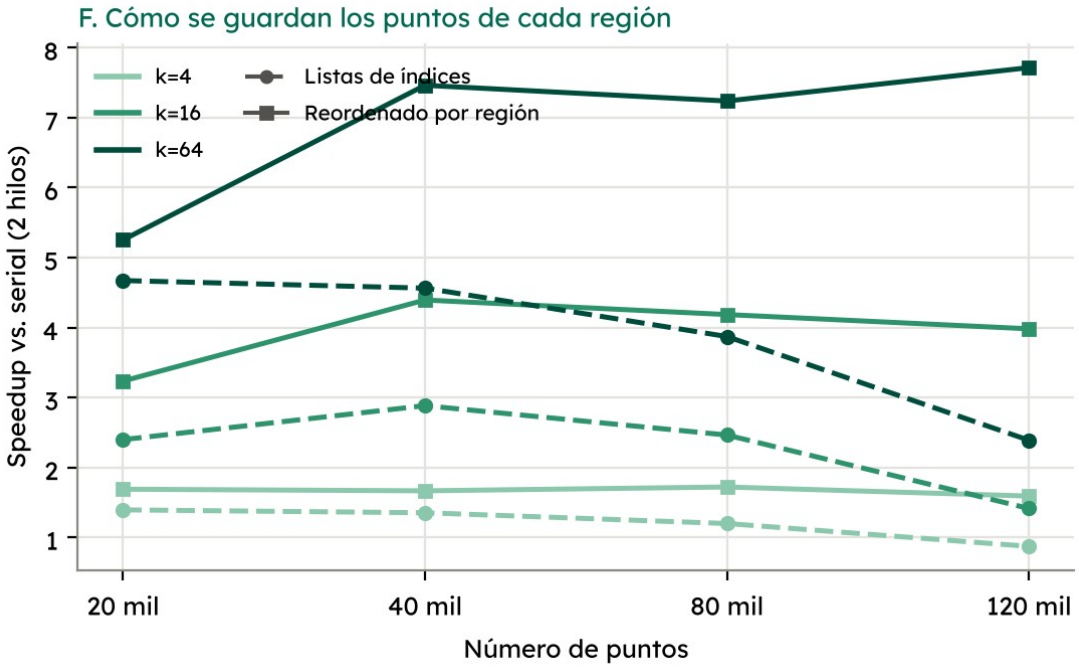


Figura 5. *Speedup de P2 con 2 hilos según el número de puntos, con listas de índices (línea punteada) o reordenado por región (línea continua).*

6.6 Tipo de dato: float o double

El código original guardaba las coordenadas como `float`. Al comparar contra `sklearn.DBSCAN` sobre los 4,000 puntos de la libreta, el programa clasificó distinto a 2 puntos: las coordenadas tienen tres decimales y hay pares de puntos a distancia exactamente `eps = 0.03`, así que el redondeo de `float` los deja de un lado o del otro de la comparación `<= eps`. `sklearn` trabaja con `double`. Con `double` las etiquetas coinciden con las de `sklearn` en todas las pruebas (sección 7), así que usamos `double` en las tres versiones.

6.7 Banderas de compilación

Tabla 8. *Tiempo medio en segundos con 40,000 puntos según las banderas de* `g++` *(5 corridas por celda).*

Banderas	Serial	P1, 2 hilos	P2 k=4, 2 hilos	P2 k=16, 2 hilos	P2 k=64, 2 hilos
`-O0`	20.64	10.91	13.29	5.34	2.99
`-O2`	1.54	0.93	1.03	0.41	0.20
`-O3`	0.81	0.49	0.56	0.20	0.12
`-O3 -march=native`	0.59	0.37	0.39	0.14	0.07

Usamos `-O2 -fopenmp` en todos los experimentos. Con `-O3 -march=native` todas las versiones mejoran entre 2 y 3 veces: la serial 2.6 veces y P2 con k=64 2.8 veces (0.20 s contra 0.07 s). Como la mejora es parecida, el speedup de P2 contra la serial casi no cambia (7.6 con `-O2` y 8.3 con `-O3 -march=native`, con k=64 y 2 hilos). Dejamos `-O2` en todos los experimentos para tener un solo caso comparable con el código escrito de forma directa como se ve en el curso, y lo mencionamos como limitación: no medimos qué parte de la mejora con `-O3` viene de vectorización y cuál de otras optimizaciones.

6.8 Resumen de las decisiones

Tabla 9. *Decisiones de diseño y los datos que las respaldan.*

Decisión	Opciones comparadas	Elegida	Evidencia
Ciclo a paralelizar en P1	Exterior / interior con reduction	Exterior	Interior con 4 hilos: speedup 0.32
Planificación en P1	static, static,16, dynamic (1, 16, 64), guided	static	Diferencias dentro de la variación entre corridas
Planificación en P2	static, dynamic,1, guided	dynamic,1	Sin diferencias medibles; se elige por robustez ante regiones desiguales
Reparto en P2	Por región / por punto	Por región	Equivalentes con k=16 y 64; el reparto por región es lo que pide el proyecto
Almacenamiento de P2	Listas de índices / bloques reordenados por región	Reordenado	Ver tabla de la sección 6.5
Tipo de dato	float / double	double	float difiere de sklearn en 2 de 4,000 puntos
Banderas	-O0, -O2, -O3, -O3 -march=native	-O2	Código directo; ver limitación

7. Cómo comprobamos que el resultado es correcto

Con `prueba_correctitud.py` se comparan P1 y P2 contra la versión serial y esta contra `sklearn.DBSCAN` (ruido y puntos core), con distintos datos: los 4,000 y 20,000 puntos de la libreta, 3,000 puntos uniformes, 2,000 puntos con muchas coordenadas repetidas y los 4,000

puntos de la libreta con `eps = 0.30` (halo de varias celdas). Para P2 se probaron k = 1, 2, 4, 8, 16, 32, 64 y también k = 7 (que no es potencia de 2), con 1, 2, 4 y 8 hilos. 190/190 comparaciones correctas

Además, el programa de experimentos guarda en cada una de las corridas si la salida de P1 y de P2 es idéntica (archivo contra archivo) a la de la versión serial con la misma entrada; en `resultados_tiempos.csv` esa información está en la columna `salida_ok`.

8. Directivas y funciones de OpenMP que se usaron

Tabla 10. *Elementos de OpenMP que aparecen en el código.*

Elemento	Dónde	Para qué
`#pragma omp parallel for`	P1, P2	Reparte las iteraciones de un ciclo entre los hilos y espera a que todos terminen al final.
`schedule(static)` / `schedule(dynamic, 1)`	P1 / P2	Cómo se reparten las iteraciones: en bloques fijos o conforme los hilos se desocupan.
Barrera implícita	Entre las fases 1 y 2	Garantiza que `core` esté completo antes de usarlo en la fase 2.
`reduction(+:vecinos)`	Solo variante P1i del estudio	Suma los contadores parciales de cada hilo (se descartó).
`omp_set_num_threads`	main	Fija el número de hilos que pide la línea de comandos.
`omp_get_wtime`	main	Mide el tiempo del algoritmo.

El uso de herramientas de IA generativa se describe en la sección final del documento de evaluación experimental.